

Лабораторна робота №1

Перша консольна програма мовою C#: введення, виведення та
математичні обчислення

Семенюка Владислава Григоровича

Група ІПЗ-21

Варіант 7

Формула

Варіант 7

$$y = \sin^2(x) + \cos^2(z)$$

```
using Lab1_Semeniuk_07;

class Program
{
    private static void ExecuteExercise1()
    {
        Console.WriteLine("Lab1 Variant 07");
        InfoValidator.StartInfoValidator();
    }

    private static void ExecuteExercise2()
    {
        MathFormula.SolveExpression();
    }

    private static void ExecuteExercise3()
    {
        Console.WriteLine("Guess a random number game!");
        Console.WriteLine("Try to guess a random number between 1 and 100!");
        GuessGame.StartGame();
        Console.WriteLine("The game is over!");
    }

    /// <summary>
    /// Entry point of the program.
    /// </summary>
    static void Main()
    {
```

```

        // Call the exercises
        ExecuteExercise1();
        ExecuteExercise2();
        ExecuteExercise3();
    }
}

namespace Lab1_Semeniuk_07;

public static class MathFormula
{
    /// <summary>
    /// Solve the 7th expression
    /// </summary>
    public static void SolveExpression()
    {
        Console.WriteLine("Formula");

        Console.Write("Enter x: ");
        // Check for a valid input and convert to double for calculation
        if (!double.TryParse(Console.ReadLine(), out double x))
        Console.WriteLine("X is not a number.\n");

        Console.Write("Enter z: ");
        // Check for a valid input and convert to double for calculation
        if (!double.TryParse(Console.ReadLine(), out double z))
        Console.WriteLine("Z is not a number.\n");

        // Prints out user's values
        Console.WriteLine("-----");
        Console.WriteLine($"X = {x}");
        Console.WriteLine($"Z = {z}");
        Console.WriteLine("-----");

        double sin = Math.Sin(x);
        double cos = Math.Cos(x);
        double y = sin * sin + cos * cos;

        // Prints out the answer
        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine($"Y = {y}");
        Console.ResetColor();
        Console.WriteLine("-----");
    }
}

```

```

}
namespace Lab1_Semeniuk_07;

// Validation Stages
internal enum InputStage {FullName, Group, Variant, Completed}

public static class InfoValidator
{
    /// <summary>
    /// Start Info Validator
    /// </summary>
    public static void StartInfoValidator()
    {
        InputStage currentState = InputStage.FullName;

        string validFullName = string.Empty;
        string validGroup = string.Empty;
        string validVariant = string.Empty;

        // Enter a validation loop where it will be asking until you enter a
        valid variant of an answer
        while (currentState != InputStage.Completed)
        {
            // Enter each validation stage
            switch (currentState)
            {
                case InputStage.FullName:
                    Console.ForegroundColor = ConsoleColor.Yellow;
                    Console.Write("Please Enter full name: ");
                    string rawFullName = Console.ReadLine()?.Trim() ??
string.Empty;

                    // If the answer is invalid make text red and print what
was wrong
                    Console.ForegroundColor = ConsoleColor.Red;

                    // Null and Empty check
                    if (string.IsNullOrEmpty(rawFullName))
Console.WriteLine("The full name is empty.\n");

                    // Length too small and too big check
                    else if (rawFullName.Length < 3 || rawFullName.Length >
50)
                        Console.WriteLine("The full name is too short or too
big.\n");

                    // Check (full name must not contain digits)
                    else if (rawFullName.Any(c => char.IsDigit(c)))
                        Console.WriteLine("Didgits are not allowed in full
name.\n");

```

```

        // The answer is valid, move on to the next validation
field
        else {
            validFullName = rawFullName;
            currentState = InputStage.Group;
        }

        break;

    case InputStage.Group:
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.Write("Please Enter group name: ");
        string rawGroup = Console.ReadLine()?.Trim() ??
string.Empty;

        Console.ForegroundColor = ConsoleColor.Red;

        // Null and Empty check
        if (string.IsNullOrEmpty(rawGroup))
Console.WriteLine("The group name is empty. \n");

        // Length too small and too big check
        else if (rawGroup.Length < 2 || rawGroup.Length > 10)
            Console.WriteLine("Thr group name is too short or too
long.\n");

        else
        {
            validGroup = rawGroup;
            currentState = InputStage.Variant;
        }

        break;

    case InputStage.Variant:
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.Write("Please Enter your variant: ");
        string rawVariant = Console.ReadLine()?.Trim() ??
string.Empty;

        Console.ForegroundColor = ConsoleColor.Red;

        // Null & Empty check
        if (string.IsNullOrEmpty(rawVariant))
Console.WriteLine("The variant name is empty \n");

        // Length too big check
        else if (rawVariant.Length > 2) Console.WriteLine("The
variant is too big!\n");

        else if (rawVariant.Any(c => char.IsLetter(c)))
            Console.WriteLine("The variant must be in digits.\n");
        else

```

```

        {
            validVariant = rawVariant;
            currentState = InputStage.Completed;
        }

        break;

    }
}
Console.ResetColor();

// Print out every collected info about the user with green color
Console.WriteLine("-----");
Console.ForegroundColor = ConsoleColor.Green;

Console.WriteLine($"Executed: {validFullName}");
Console.WriteLine($"Group: {validGroup}");
Console.WriteLine($"Variant: {validVariant}");

Console.ResetColor();
Console.WriteLine("-----");
}
}

namespace Lab1_Semeniuk_07;
using System;

enum GameStatus {Initial, TooBig, TooSmall, Guessed}

public static class GuessGame
{
    private static Random? _random;
    private static int _guessNumber;
    private static GameStatus _gameStatus;
    private static int _trys;

    /// <summary>
    /// Init constructor with default values
    /// </summary>
    static GuessGame()
    {
        _gameStatus = GameStatus.Initial;
        _trys = 0;
        GenerateNewGuessedNumber(1, 100);
    }

    /// <summary>
    /// Generates a random number between (min and max values)

```

```

    /// </summary>
    /// <param name="min"></param>
    /// <param name="max"></param>
    static void GenerateNewGuessedNumber(int min, int max)
    {
        _random = new Random();
        _guessNumber = _random.Next(min, max + 1);
    }

    /// <summary>
    /// Print Status-Bar for the game
    /// </summary>
    /// <param name="userNumber"></param>
    /// <param name="status"></param>
    /// <param name="trys"></param>
    private static void PrintGameStatusBar(int userNumber, GameStatus status,
int trys)
    {
        Console.WriteLine(new string('-', 66));

        // Red color for wrong answers
        Console.ForegroundColor = ConsoleColor.Red;
        // Green for the correctly guessed number
        if (status == GameStatus.Guessed) Console.ForegroundColor =
ConsoleColor.Green;

        Console.WriteLine($"    \tYour number: {userNumber} | Game Status:
{status} | Trys: {trys,-10}\t");
        Console.ResetColor();
        Console.WriteLine(new string('-', 66));
    }

    /// <summary>
    /// Start the Game with infinite loop until a user finds the guessed
number
    /// </summary>
    public static void StartGame()
    {
        while (true)
        {
            Console.Write($"Enter a number: ");

            // Check if a user entered a valid value (must be only int)
            if (int.TryParse(Console.ReadLine(), out int userNumber))
            {
                _trys++;
                if (_guessNumber == userNumber)
                {
                    _gameStatus = GameStatus.Guessed;
                    PrintGameStatusBar(userNumber, _gameStatus, _trys);
                    // Exit the loop if a user guesses the number
                    break;
                }
            }
        }
    }

```

```
    }

    if (_guessNumber > userNumber)
    {
        _gameStatus = GameStatus.TooSmall;
        PrintGameStatusBar(userNumber, _gameStatus, _trys);
    }else if (_guessNumber < userNumber)
    {
        _gameStatus = GameStatus.TooBig;
        PrintGameStatusBar(userNumber, _gameStatus, _trys);
    }
}
else
{
    Console.WriteLine("You entered invalid value!");
}

}

}
```

Дві пари тестових значень і отримані результати

Lab1 Variant 07

Please Enter full name:

The full name is empty.

Please Enter full name: Vlad Semeniyuk

Please Enter group name: d

The group name is too short or too long.

Please Enter group name: IPZ-21

Please Enter your variant: 7

Executed: Vlad Semeniyuk

Group: IPZ-21

Variant: 7

Formula

Enter x: 5

Enter z: 5

X = 5

Z = 5

Y = 0,999999999999999999

```
Guess a random number game!
Try to guess a random number between 1 and 100!
Enter a number: 50
-----
    Your number: 50 | Game Status: TooBig | Trys: 1
-----
Enter a number: 25
-----
    Your number: 25 | Game Status: TooSmall | Trys: 2
-----
Enter a number: 30
-----
    Your number: 30 | Game Status: TooSmall | Trys: 3
-----
Enter a number: 40
-----
    Your number: 40 | Game Status: TooSmall | Trys: 4
-----
Enter a number: 45
-----
    Your number: 45 | Game Status: TooSmall | Trys: 5
-----
Enter a number: 48
-----
    Your number: 48 | Game Status: TooBig | Trys: 6
-----
Enter a number: 46
-----
    Your number: 46 | Game Status: Guessed | Trys: 7
-----
The game is over!
```

Додадкове завдання зверху
Щі використовувався лише для ресьорчу